Agentic AI for Business and FinTech (FTEC5660)
Individual Homework 01 Due date: 30 January, midnight
Name: YANG Jinyue Student ID:1155242966

1. Introduction

Based on Gemini API, this project develops core functions, including processing batch receipt images, recognizing user intent correctly (e.g., total consumption amount or original price), and billing calculation. Specifically, it takes common edge cases in practical situations into consideration (e.g., rounding discrepancies and non-standard receipt layouts) for the accuracy of final output.

2. System Architecture

It has a pipeline: intent classification → data extraction → accurate computation. This modular design logically separates each step, so the system is more reliable.

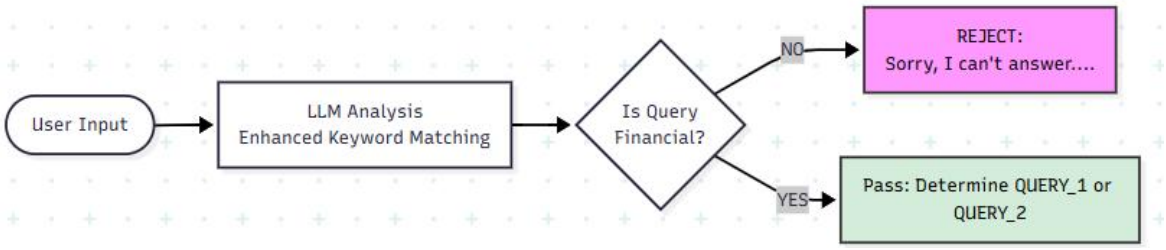
Intent classification: classify_query

• This part makes a semantic analysis of natural language input.

•With enhanced keyword matching, it can recognize standard terms like *Total, Original Price* and other terms like *Gross amount, Cost before discount*.

• There is a rejection framework, through which it can reject irrelevant questions(e.g., dates, weather) and automatically reply with "Sorry, I can't answer the query".

```
prompt = """Classify the user query into one of these categories:
- QUERY_1: total paid, amount spent, final sum, ending total.
- QUERY_2: pre-discount total, cost before discount, original price, price without savings.
- REJECT: Any other questions (e.g., weather, company name, items).
```



Data extraction

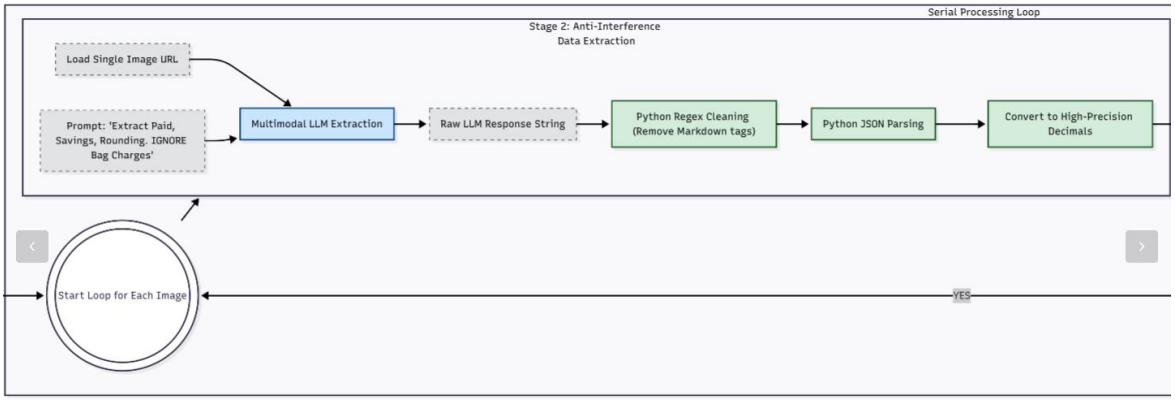
• It can extract raw data through LLM for visual perception (receipt reading) and impose constraints on the output results.

• There is a key problem—plastic bag fees are easily mistreated as discounts—which leads to errors in query 2 during the evaluation part. So I revise the prompts to explicitly state 'Ignore plastic bag fees' to make sure that these charges won't affect discount calculations.

• LLM models often produce formatting errors (e.g., Markdown back quotes). The code includes a regular expression cleanup step to remove these tags before parsing, preventing JSONDecodeError during batch processing.

```
prompt = """You are a senior auditor. Look at the SUMMARY section (bottom part) of the receipt.
Identify these values:
1. 'subtotal' (the subtotal/total before discounts).
2. 'savings' (total savings/discounts). IGNORE plastic bag charges.
3. 'paid' (final amount paid via Octopus/Visa/Total).
4. 'rounding' (any rounding adjustment, e.g., 0.05 or -0.02). If none, use 0.
```

Output ONLY JSON: {'subtotal': float, 'savings': float, 'paid': float, 'rounding': float}."""



Accurate computation

Computation uses Python's Decimal library instead of standard floating-point types, which avoids floating-point errors and ensures results are rounded up to the nearest whole number through ROUND HALF_UP method.

3. Key Decisions

Separation of Concerns

Decouple data extraction from computation. LLM is only used to identify and replicate numbers, without arithmetic operations. Python copes with addition and subtraction, which can avoid the risk of computational errors in LLM.

Stable Processing

During testing, I notice that Google API frequently triggered rate limits when sending parallel requests. So in the agent_pipeline, I use a serial processing approach to process images one by one. It slightly increases runtime but guarantee the success rate.

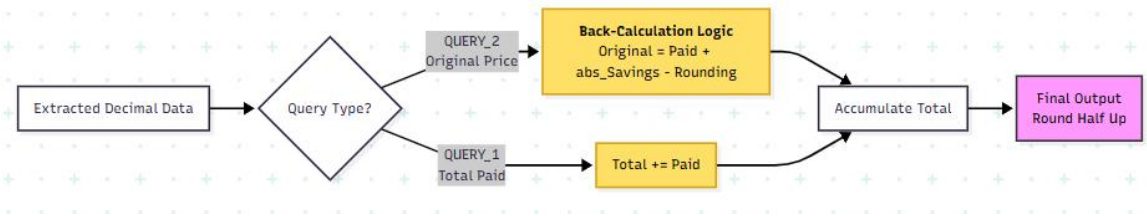
Edge cases

Receipts often include 'rounding' (e.g., $\pm$ 0.05). Simply adding up the subtotals often leads to discrepancies. So I use the logic:

Original price=Payment amount + |Discount| - Rounding.

Based on the explicit payment amount on the receipt, reconstruct the pre-discount price by reverse calculation.

```
if q_type == "QUERY_2":
    # Formula: Original = Final Payment + |Discounts| - Rounding Adjustment
    # Using abs() ensures positive/negative discounts are handled correctly
    original = paid + abs(savings) - rounding
    total_result += original
```



4. Result

By setting specific prompt constraints (e.g., ignoring bag costs) and precise decimal operations, the system can effectively filter out noise and reliably achieve automated receipt calculations and answer questions.

```
── Final Precision Audit: QUERY_1 ──
File: receipt1.jpg | Paid: 394.7
File: receipt2.jpg | Paid: 316.1
File: receipt3.jpg | Paid: 140.8
File: receipt4.jpg | Paid: 514.0
File: receipt5.jpg | Paid: 102.3
File: receipt6.jpg | Paid: 190.8
File: receipt7.jpg | Paid: 315.6
Passed! Your calculated total is: 1974.3

── Final Precision Audit: QUERY_2 ──
File: receipt1.jpg | Paid: 394.7 | Savings: 85.48 | Rounding: -0.02 | Total: 480.20
File: receipt2.jpg | Paid: 316.1 | Savings: 76.09 | Rounding: -0.01 | Total: 392.20
File: receipt3.jpg | Paid: 140.8 | Savings: 19.22 | Rounding: -0.08 | Total: 160.10
File: receipt4.jpg | Paid: 514.0 | Savings: 76.71 | Rounding: -0.09 | Total: 590.80
File: receipt5.jpg | Paid: 102.3 | Savings: 5.39 | Rounding: -0.01 | Total: 107.70
File: receipt6.jpg | Paid: 190.8 | Savings: 30.31 | Rounding: -0.09 | Total: 221.20
File: receipt7.jpg | Paid: 315.6 | Savings: 80.36 | Rounding: -0.04 | Total: 396.00
Passed! Your calculated total is: 2348.2
```

Figure: Execution Log from Google Colab